

HW4 Report: Facial Emotion Classification using SVM with HoG and Deep Features

Johnpaul Mbagwu

System T: Hand-Crafted Features (HoG) + SVM

Approach:

Images were accessed from Google Drive and converted to grayscale regardless of format. Histogram of Oriented Gradients (HoG) features were extracted using `skimage.feature.hog`. These features were then used to train a linear Support Vector Machine (SVM), with an 80-20 split for training and testing.

Performance:

- **Training Accuracy:** 0.67
- **Testing Accuracy:** 0.64

Q2: Is HoG + SVM suitable for emotion classification? Why or why not?

While this method is computationally efficient, it may not fully capture the subtle and complex variations in facial expressions, as HoG mainly emphasizes edge and texture features. This limitation likely contributes to the modest 64% test accuracy, indicating that it may not be optimal for nuanced tasks like emotion recognition.

System G: Deep Features from ResNet18 + SVM

Approach:

Images were resized and normalized to fit the input requirements of ResNet18. A pre-trained ResNet18 (with its classification head removed) was used as a fixed feature extractor. The resulting deep features were classified using a linear SVM.

Performance:

- **Training Accuracy:** 0.90
- **Testing Accuracy:** 0.77

Q4: Is using a pre-trained model with SVM effective? Why or why not?

Yes, this approach is quite effective. Pre-trained deep models like ResNet18 capture complex, high-level visual representations, improving class separability in emotion

detection tasks. The 77% test accuracy reflects its ability to generalize well without needing additional model training.

Q5: Which system performed better and why?

System G clearly outperformed System T. The 77% test accuracy achieved using deep features from ResNet18 shows the strength of learned representations compared to handcrafted ones. ResNet18’s hierarchical feature extraction (from edges to object-level understanding) gives it a significant edge in capturing the subtleties of facial emotions.

Summary Table

System	Feature Type	Train Accuracy	Test Accuracy
System T	HoG (Hand-crafted)	0.67	0.64
System G	ResNet18 (Deep features)	0.90	0.77

Code Used

```
#!/usr/bin/env python3

# -*- coding: utf-8 -*-

"""
Facial Expression Classification System

Compare traditional (HoG+SVM) vs deep learning (ResNet18+SVM) approaches
"""

# ----- Setup Environment -----

# Mount cloud storage

from google.colab import drive

drive.mount('/content/drive')

# Install required packages
```

```
!pip install scikit-image torch torchvision --quiet
```

```
# ----- Import Dependencies -----
```

```
# Image processing
```

```
from skimage.feature import hog
```

```
from skimage import color, io
```

```
from PIL import Image
```

```
# Machine learning
```

```
from sklearn.svm import SVC
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
```

```
# Deep learning
```

```
import torch
```

```
from torchvision import models, transforms
```

```
# System utilities
```

```
import os
```

```
import glob
```

```
import numpy as np
```

```
# ----- Configure Paths -----
```

```
# ⚠ Update this path to your dataset location
```

```
dataset_path = '/content/drive/My Drive/Happy_Angry_Dataset_3K/dataset'
```

```

# ----- Data Preparation -----

# Collect all image files

image_extensions = ('*.jpg', '*.jpeg', '*.png')

image_paths = []

for ext in image_extensions:

    image_paths.extend(glob.glob(os.path.join(dataset_path, ext)))

print(f"[STATUS] Found {len(image_paths)} images")


# ----- Traditional Approach: HoG + SVM -----

hog_features = []

class_labels = []


for path in image_paths:

    # Extract label from filename

    fname = os.path.basename(path).lower()

    if 'happy' in fname:

        lbl = 0

    elif 'angry' in fname:

        lbl = 1

    else:

        continue # Skip unlabeled images


try:

    # Process image

    img = io.imread(path)

    gray_img = color.rgb2gray(img) if img.ndim == 3 else img

```

```

# Extract HOG features

features = hog(gray_img,
               orientations=8,
               pixels_per_cell=(16, 16),
               cells_per_block=(1, 1),
               visualize=False)

hog_features.append(features)

class_labels.append(lbl)

except Exception as error:

    print(f'Error processing {path}: {error}')


# Train/test split and model evaluation

if class_labels:

    X_train, X_test, y_train, y_test = train_test_split(
        np.array(hog_features),
        np.array(class_labels),
        test_size=0.2,
        random_state=42
    )

    svm_classifier = SVC(kernel='linear')

    svm_classifier.fit(X_train, y_train)

    train_accuracy = svm_classifier.score(X_train, y_train)

    test_accuracy = svm_classifier.score(X_test, y_test)

```

```

        print(f'HoG+SVM Performance | Train: {train_accuracy:.2f}, Test: {test_accuracy:.2f}')
    else:

        print("No valid images processed for HoG pipeline")


# ----- Deep Learning Approach: ResNet18 + SVM -----
# Initialize feature extractor
resnet_model = models.resnet18(weights='IMAGENET1K_V1')
feature_extractor = torch.nn.Sequential(*list(resnet_model.children())[:-1])
feature_extractor.eval()


# Image preprocessing pipeline
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])


# Feature extraction
cnn_features = []
cnn_labels = []


for path in image_paths:
    # Extract label from filename
    fname = os.path.basename(path).lower()

```

```

if 'happy' in fname:

    lbl = 0

elif 'angry' in fname:

    lbl = 1

else:

    continue


try:

    # Process image

    img = Image.open(path).convert('RGB')

    tensor_img = preprocess(img).unsqueeze(0)


    # Extract features

    with torch.no_grad():

        features = feature_extractor(tensor_img).flatten().numpy()


    cnn_features.append(features)

    cnn_labels.append(lbl)

except Exception as error:

    print(f"Error processing {path}: {error}")


# Model training and evaluation

if cnn_labels:

    X_train, X_test, y_train, y_test = train_test_split(

        np.array(cnn_features),

        np.array(cnn_labels),

```

```

        test_size=0.2,
        random_state=42
    )

    svm_model = SVC(kernel='linear')
    svm_model.fit(X_train, y_train)

    train_acc = svm_model.score(X_train, y_train)
    test_acc = svm_model.score(X_test, y_test)
    print(f'ResNet18+SVM Performance | Train: {train_acc:.2f}, Test: {test_acc:.2f}')
else:
    print("No valid images processed for ResNet pipeline")

# ----- Performance Report -----
print("\nFinal Comparison:")
if class_labels:
    print(f'Traditional (HoG+SVM): Test Accuracy {test_accuracy:.2f}')
if cnn_labels:
    print(f'Deep Learning (ResNet18+SVM): Test Accuracy {test_acc:.2f}')

```